Rendered 2026-10-16

BIP	TBD
Layer	Consensus (soft fork)
Title	Value-Scaled Spend Delay (VSD) for Large-Value UTXOs
Authors	xfleeeee
Comments-Summary	No comments yet.
Comments-URI	TBD
Status	Draft
Type	Standards Track
Created	2026-10-14
Last-updated	2026-10-16
License	BSD-3-Clause
Requires	TBD

> **Changelog (v2.0, 2026-10-16).** This version promotes the **age-aware delay factor** (the `W(age)` discount table, `F` floor, and `D_cap` cap) from a future-work note (§Design evolution, Step 7, in v1.0) into a **first-class part of the specification**. The top-level rule is now `D(v, age)` rather than `D(v)`. All v1.0 values that quoted "~14 days" for the `>= 1000 BTC` tier or "~7 days" for the `100 – 1000 BTC` tier are **corrected**: `D = 20160` blocks is ~140 days (~4.7 months) and `D = 10080` blocks is ~70 days (~2.3 months). The "base VSD" (v1.0) is retained as the special case `W(age) ≡ 1`.

Value-Scaled Spend Delay (VSD)

Abstract

This proposal introduces a single consensus rule: **spending a confirmed UTXO whose value meets or exceeds a threshold requires a relative height delay that scales with the UTXO's value and is discounted by the UTXO's age (the time since its confirmation).** Small transactions are unaffected. The rule is a pure soft fork (it only *reduces* the set of valid transactions), changes no data format, adds no new cryptographic primitive, and introduces no new opcode. Its purpose is to convert the confirmation of a large-value transfer from "within one block (~10 minutes)" into "after N blocks", thereby giving a legitimate holder a discovery-and-response window if a key is compromised — without imposing a uniform cost on day-to-day large-value flows.

The age discount (`W(age)`) makes the protection *proportional to the risk profile* of the UTXO being spent: a UTXO that has sat dormant for years (a profile consistent with lost/dormant keys — the highest-risk category) carries the full delay; a UTXO the holder has been actively moving recently (a profile consistent with a live, responsive holder) carries a proportionally shorter delay. The full delay for the highest tier is preserved as the floor for any UTXO that has matured; a fresh UTXO at the highest tier is delayed by ~7 days rather than ~4.7 months, reducing the operational cost to legitimate active holders by roughly a factor of 20.

This proposal is intentionally **partial and conservative**. It is not a substitute for quantum-resistant signatures (e.g. a future P2MR/PQ-signature soft fork) or for public-key-hiding output types (P2WPKH/P2TR). It is an additive, behavior-layer defense intended to buy time in the common, recoverable scenario: *a key is stolen or broken, but the holder is alive and monitoring*.

Problem Statement

The threat is concrete, not hypothetical

By mid-2026, approximately **34% of the Bitcoin supply (~6.6 million BTC)** sits on addresses whose public keys are already exposed on-chain (P2PKH/P2PK outputs that have been spent at least once, revealing the public key in the transaction witness). A cryptographically relevant quantum computer (CRQC) running Shor's algorithm can derive the private key from any exposed public key, making all funds on those addresses spendable by the attacker. This is not a "when" question but a "when can we afford to wait" question.

Even without a CRQC, the more immediate and far more common threat is **key theft**: online wallets hacked, cloud-storage breaches, malware extracting seed phrases, employees exfiltrating keys from custodial platforms. In every case the attacker ends up holding the same private key the victim once held.

Why Bitcoin has no built-in defense against this

Bitcoin's UTXO model provides **no revocation mechanism and no spend-gating mechanism**:

1. Once a private key is compromised, the attacker can create a validly-signed transaction spending *any* UTXO controlled by that key.
2. That transaction enters the mempool and is typically mined within **one block (~10 minutes)** — too fast for any human or automated system to notice and respond.
3. There is no consensus-level rule that slows down, flags, or requires approval for large-value transfers. A 1,000 BTC transfer confirms at the same speed as a 1 sat transfer.

The net effect: **a theft of a large UTXO is irrevocable within ~10 minutes**, before the victim, their monitoring systems, or the broader community can observe it and mount any response.

What a "cooling-off" mechanism must actually buy

The core value a "cooling-off" / "冷静期" mechanism must provide is **time between the theft attempt and its irrevocable confirmation**. Everything else — governance, multi-sig, insurance — is secondary. Concretely, the delay buys three distinct things depending on the holder's situation:

Holder situation	What the delay buys
Online, has a backup (hardware wallet, seed phrase, multisig)	Time to notice the pending theft in the mempool and win the fee race to re-spend the UTXO with a legitimate key

Holder situation	What the delay buys
Offline / unaware	Converts "stolen in 10 minutes" into "stolen after N days", giving monitoring systems or a delayed-notice holder a chance to intervene
No holder (lost keys, Satoshi P2PK)	A lengthened observation window for the community to coordinate a response, if one is ever deemed appropriate

The proposal is designed around the first row — *the common, recoverable scenario where the holder is alive and can respond* — while providing diminishing (but nonzero) value for the second and third rows.

Rationale

The core challenge: the attacker has the same key

The most important insight that shaped this design is deceptively simple: **when a private key is compromised, the attacker holds the same key as the victim.** Any mechanism that relies on the victim "objecting," "signing a cancellation," or "locking their own UTXO" is inherently broken, because the attacker can forge an identical signature.

This single fact rules out an entire class of otherwise-intuitive designs:

- **Opt-in locking**: the holder "locks" their UTXO. But the attacker (holding the same key) can simply *not* lock, or can unlock.
- **On-chain objection / veto**: the holder "objects" to a spend. But the attacker's signature is indistinguishable from the holder's — the objection is forgeable.
- **Mandatory multisig for new outputs**: breaks backward compatibility and the UTXO model for all existing single-key holdings.

The only mechanism that works against a same-key attacker is one that is **mandatory, consensus-enforced, and cannot be bypassed by the key-holder** — i.e., a rule the attacker cannot sign away.

The gap

Bitcoin UTXOs are non-revocable. A "cancellation" of a transfer is not a real revoke; it is a *competing* spend of the same UTXO, funded by the same private key. When a private key is compromised (stolen, or derived by a cryptographically relevant quantum computer from an exposed public key), the attacker holds the same key as the victim. A transfer of a large UTXO confirms in ~10 minutes, which is typically too fast for the victim to notice and respond.

The core value a "cooling-off" / "冷静期" mechanism must buy is **time between the theft attempt and its irrevocable confirmation**. Everything else is secondary.

Why this shape

- The primitive needed already exists and is battle-tested: **relative height locking (OP_CSV, BIP-68)**, active since 2017. What this BIP adds is not the primitive but a **mandatory, value-scaled (and age-discounted) application** of it

at the consensus layer — so it cannot be opted out of by a sender who simply sets `nSequence = 0`.
- It is a **soft fork**: the new validity set is a strict subset of the old. Old nodes continue to validate the chain new nodes build; they merely do not understand that certain large-value spends were deliberately not included.
- It **only changes behavior logic**, per the design constraint: no new address type, no new output format, no new signature algorithm, no governance layer. The only new thing is one conditional rule keyed on a UTXO's confirmed value and its confirmation height (both already present in the UTXO set).
- It **does not disrupt normal transactions**: below a threshold the delay is zero; above the threshold, an actively-held UTXO (fresh age) is delayed by a small number of days, not months.

Design evolution: how VSD was arrived at

The design above is the endpoint of a series of deliberate narrowing steps. Each step was driven by a specific problem with the previous candidate. Reading the rejected alternatives makes the final shape intelligible — and documents *why* each design constraint is there.

Step 1 — The naive "cooling-off" idea. The initial proposal was: give large-value addresses a mandatory delay, and require that no "objection" be filed on-chain within the delay window before the transfer completes. On the surface this looks like a strong guarantee: a victim has a "grace period" to cry foul.

Step 2 — The same-key attack breaks objections. As soon as the same-key threat model (§The core challenge) was applied, the objection mechanism collapsed: *an objection must be signed by the key-holder — and the attacker holds that key.* The attacker can simply not object, or object-and-then-spend, or sign the objection with the stolen key and then spend with the same key. The objection channel is no more secure than the spend channel; adding it provides no new protection. This ruled out the entire "community veto / on-chain objection" design family.

Step 3 — "What if the objection requires a *different* key?" A natural refinement: require a *recovery key* (a second key the holder controls but the attacker presumably does not) to file an objection. This works *if and only if* the holder has already set up and safeguarded a recovery key, and *fails* in exactly the scenario the proposal targets — a holder who did not, or whose recovery key was also compromised. Forcing a recovery-key requirement on all large UTXOs would also change the output format (a new script type), violating the "behavior-only change" constraint. It was set aside as a *holder-optional enhancement*, not a consensus rule.

Step 4 — "What if the delay is triggered by value, with no objection at all?" Stripping out the objection mechanism leaves a clean primitive: *any spend of a UTXO above a value threshold requires waiting N blocks before it can be confirmed.* This is exactly a mandatory OP_CSV with a value-keyed parameter. No new primitives, no new formats, no governance — just a consensus rule. This is the core of VSD.

Step 5 — "Why not a flat delay for all large UTXOs?" A single delay (e.g. "every UTXO ≥ 1 BTC waits 7 days") penalizes a legitimate 5 BTC payment the same as a 5,000 BTC theft. A value-scaled delay makes the protection *proportional to the risk*: a 5 BTC transfer gets a short delay; a 5,000 BTC transfer gets the maximum. This is what makes the rule acceptable to the economy without a hard-fork-level disruption.

Step 6 — "Why a soft fork, not a hard fork?" The new rule *rejects transactions the old rule accepted* (early large-value spends). A soft fork can express this: the new valid set is a subset of the old valid set. Old nodes continue to accept the chain; they just don't see (or care about) the withheld early spends. A hard fork would require every node to upgrade simultaneously — a coordination and centralization risk this proposal deliberately avoids.

Step 7 — "What about the usability cost for legitimate large-value holders?" A legitimate whale who must send 100 BTC today now has to wait 10080 blocks (~2.3 months) under the base value-only rule — a real operational cost. This is the single most serious objection to VSD, and the reason this proposal evolved from a value-only delay (`D(v)`) into the current **value-and-age delay** (`D(v, age)`), detailed in the next step.

Step 8 — "Why UTXO age, not address activity?" The most natural intuition is to shorten the delay for *addresses* the holder has been actively using. This fails the same-key threat model: address activity is observable from the mempool, and an attacker who has stolen a key can simply *refresh* a dormant address with a 1-sat transaction to make it look active, then immediately spend the large dormant UTXOs with a short delay. The attacker can manipulate the activity signal; the holder cannot prevent it.

The **UTXO's own age** — the number of blocks since *that specific UTXO* was confirmed — is a fundamentally different signal: it is derived from the UTXO's confirmation height, which is immutable chain data. An attacker who steals a dormant 5,000-BTC UTXO cannot make it younger; the only way to reduce its delay is to spend it (which incurs the full dormant-age delay) and re-create a fresh child UTXO (which then has its own, shorter delay). The age signal is therefore **tamper-resistant against the same-key attacker** in a way that address activity is not.

This is the design insight that shaped v2.0: use the UTXO's own age as the discount factor, not the address's recent activity.

Step 9 — "How to express the discount: step table or linear ramp?" Two candidate forms were considered: - **Step table** (adopted): a small number of age tiers with a fixed discount per tier. Pros: trivially auditable, each row is a tunable consensus parameter, easy to read in a BIP. Cons: discontinuities at tier boundaries (a UTXO at age 143 vs. age 144 has a different discount). - **Linear ramp**: a continuous function of age. Pros: smooth. Cons: harder to audit, harder to tune per-segment, and the BIP-2 process is friendlier to step functions (the existing value tier table is already a step function, so the age table matches the house style).

The step table was adopted. The discontinuities are acceptable: they are a property of the *draft* values, not of the mechanism, and the community can tune or smooth them during review.

Step 10 — "What about the floor?" A fresh UTXO at the highest value tier, under a pure `Dv × W` product, would be delayed by `20160 × 0.05 = 1008` blocks (~7 days) — already a meaningful window. But for the *lowest* value tier (`1 – 10 BTC` , `Dv = 144`), the product `144 × 0.05 = 7` blocks (~1 hour) is too short to be useful: a 5-BTC theft would be confirmed in ~1 hour, leaving almost no time to respond. A **floor** `F` (a minimum delay applied to *any* UTXO at or above the threshold, regardless of age) addresses this. The draft value `F = 144` blocks (1 day) is a single scalar, decoupled from the value and age axes, so the community can tune it independently. In practice, with the draft value table, `F` only binds for the `1 – 10 BTC` tier (where `Dv × W_min = 7 < 144`); for all higher tiers the product already exceeds `F` .

What this evolution shows: every design constraint in this proposal — "behavior-only change," "soft fork," "value-scaled," "age-discounted (not address-activity-discounted)," "step table," "mandatory not opt-in" — is the *direct consequence* of a specific problem with a more intuitive alternative. The design is not arbitrary; it is the intersection of "works against a same-key attacker," "does not require a hard fork or new data format," and "does not impose a uniform cost on active large-value holders."

A concrete threat scenario, walked through

Consider the following realistic scenario to make the mechanism tangible:

1. **Alice** holds 500 BTC in a P2WPKH UTXO on a hardware wallet, confirmed at height 790,000 (~7 months ago, i.e. dormant). She also runs a small online wallet with 0.3 BTC (daily spending).
2. **Day 1:** An attacker compromises Alice's online wallet's server and extracts the 0.3 BTC (fast, unnoticed, small). They also learn that Alice has a separate hardware-wallet account — but they do *not* have the hardware wallet's seed.
3. **Day 40:** A CRQC operator (or a very patient social engineer who has finally extracted the hardware wallet's seed) obtains Alice's 500 BTC key. They create a transaction moving the 500 BTC to a fresh attacker-controlled address, paying a high fee. The current chain tip is at height 800,000, so the UTXO's age is 800,000 − 790,000 = 10,000 blocks (~23 days).
4. **Without VSD:** The 500 BTC transaction is mined in the next block (~10 minutes). Alice notices 30 minutes later. By then the 500 BTC is confirmed to the attacker's address, which will sweep it through mixing services. **Recovery is effectively impossible.**
5. **With VSD v2.0 (draft values):** The 500 BTC UTXO is worth 500 BTC (100–1000 tier, `Dv = 10080`). Its age of 10,000 blocks falls in the `144 – 10079` age tier → `W = 0.15`. The required delay is `D = min(20160, max(144, round(10080 × 0.15))) = 1512` blocks (~10.5 days). The attacker's transaction enters the mempool but **cannot be mined until 1512 blocks after the UTXO's confirmation height** (i.e. at height 790,000 + 1512 + 1 = 791,513, which has already passed — so the transaction is *already* matured at the time of the attack).

Correction: because the UTXO is already matured (its age exceeds the delay), the attacker can spend it immediately. The age discount *does not protect* a UTXO that the attacker can reach at a height where it has already matured. This is the honest limit of the age-discount design: it shortens the delay for *fresh* UTXOs (where the holder is presumably active), but for a *dormant* UTXO that has already accumulated enough age to be "mature," the delay is the full tier value and the attacker can still spend it the moment they obtain the key.

The age discount's real value is in the *next* scenario: suppose Alice *had* been actively moving her 500 BTC (e.g. a partial re-spend 20 days ago, leaving a fresh 500 BTC UTXO at age ~20,000 blocks). The attacker's transaction would then face a delay of `D = round(10080 × 0.35) = 3528` blocks (~24.5 days) — a much longer window for Alice to notice and respond than the base value-only rule's 10,080 blocks (~70 days) would provide *only if* the UTXO were dormant. In practice, the age discount helps *active* holders (who can afford a shorter delay) and *does not* shorten the protection for *dormant* UTXOs (which get the full tier delay). The net effect is to make the rule acceptable to active large-value holders without weakening the protection for the high-risk dormant category.

6. **Alice's response window (active-holder case):** In the scenario where the 500 BTC UTXO is fresh (recently re-spent by Alice), Alice has ~24.5 days of real time to: (a) confirm the theft is real, (b) locate her backup hardware wallet / seed phrase, (c) construct a competing transaction that re-spends the 500 BTC UTXO to a fresh, attacker-unreachable address, (d) pay a higher fee to get it mined before the attacker's transaction matures. Because both transactions reference the same UTXO, only one can succeed.
7. **Outcome with VSD (active-holder case):** The 500 BTC is moved to Alice's new address. The attacker's transaction is invalidated. The 0.3 BTC already stolen on Day 1 is unrecoverable (below threshold, confirmed in minutes) — but the 500 BTC is saved.

This is the exact scenario the threat model's first row describes: *key stolen, holder online and has a backup*. VSD converts a 10-minute window (no response possible) into a multi-day window (response very possible). The 0.3 BTC loss is the accepted cost of protecting the large holdings; it is below the threshold and outside the design's scope.

Threat model (in and out of scope)

Scenario	VSD helps	What the delay buys	Why
Key stolen/broken, holder **online & has a backup**	✅ primary target	~days to notice + win the fee race to re-spend with a legitimate key	holder observes the pending large spend in mempool and re-spends the UTXO; only one of the two competing spends can confirm
Key stolen/broken, holder **offline / unaware**	⚠️ partial	converts "stolen in 10 min" → "stolen after N days", so a delayed notice still has a chance	attacker must wait the full delay before the theft is irrevocable; any delayed response can still intervene
No holder (e.g. lost keys / Satoshi P2PK)	⚠️ observation only	a lengthened window for the community to observe and (if ever appropriate) coordinate	no one can race; VSD only slows the confirmation so a response is not pre-empted
HD wallet (one key → many addresses)	⚠️	buys time on the *specific* UTXO being spent, not the whole key	racing one UTXO does not protect the others; the holder must still rotate the key chain
Custodial / exchange-held coins	⚠️	buys the custodian time to detect an internal key-compromise and respond	depends on the custodian upgrading, monitoring, and having a recovery path

VSD is designed to be **honest** about this: it is a "best-effort" defense, a last time window layered *under* the two root-cause fixes (key-hiding outputs and post-quantum signatures). It is not claimed to be a complete solution. The first row is where it delivers its full value; the remaining rows get diminishing but nonzero value.

Known usability tradeoff (documented for reviewers): the mandatory delay imposes a real cost on *legitimate* large-value holders. The age discount (`W(age)`) reduces this cost substantially: a fresh 1000 BTC UTXO is delayed by ~7 days rather than ~4.7 months (a 20× speedup). This is the proposal's most serious objection *partially* addressed by (a) keeping all delay values **draft and tunable** (§Open parameters), and (b) the age-discount mechanism itself (§Specification, "Age discount factor"). A **recovery-key exemption** (a holder who has registered a secondary recovery key gets an additional discount) remains a follow-up refinement.

Specification

Definitions

- **Value.** The number of satoshis in a UTXO's `value` field (internal unit: satoshis; 1 BTC = 100,000,000 sats).
- **`outpoint_height(O)`** — the height of the block that confirmed UTXO `O` (the block containing the creating transaction). For an unconfirmed (mempool-only) previous output, the outpoint is not a confirmed UTXO and is *not* subject to this rule (see "Unconfirmed previous outputs").
- **Threshold `T`** — the minimum UTXO value (in satoshis) at which the delay applies. A UTXO with `value < T` is not subject to the delay. Draft: `T = 100,000,000` sats (1 BTC).
- **Value-delay function `Dv(v)`** — a monotonic, non-decreasing step function mapping a UTXO value to a base delay in number of blocks. See table below.

- **Age of a UTXO, `age(O, H)`** — the number of blocks by which the spending height `H` exceeds the UTXO's confirmation height: `age(O, H) = H - outpoint_height(O)`. (For a UTXO confirmed at height `c` and spent at height `H`, the age is `H - c` blocks.)
- **Age-discount factor `W(age)`** — a monotonic, non-decreasing step function mapping a UTXO's age to a discount factor in `[0, 1]`. Fresh UTXOs (small age) get a small factor (short delay); mature UTXOs (large age) get a factor of 1 (full delay). See table below.
- **Floor `F`** — a minimum delay (in blocks) applied to *any* UTXO at or above the threshold, regardless of value tier or age. Draft: `F = 144` blocks (1 day).
- **Cap `D_cap`** — a maximum delay (in blocks) that no UTXO may ever require, regardless of value or age. Draft: `D_cap = 20160` blocks (~4.7 months).
- **Delay function `D(v, age)`** — the composite delay function: `if v < T: D = 0 else: D = min(D_cap, max(F, round(Dv(v) × W(age))))`
- **Spending height `H`** — the height of the block that includes the spending transaction (for block validation), or the chain tip height (for mempool eligibility).

The rule

For each transaction input referencing a **confirmed** previous output `O`, with spending height `H`:

```
if O.value >= T:
    age = H - outpoint_height(O)
    D = min(D_cap, max(F, round(Dv(O.value) * W(age))))
    required = outpoint_height(O) + D +
    the spending transaction may only be included in a block of height H
    such that H >= required.
```

`required` is the **earliest block height at which a spend of the UTXO is mineable**. The `+1` is not a VSD invention — it is the standard Bitcoin rule that a UTXO cannot be spent in the same block in which it is confirmed. For a zero-delay (`D=0`) UTXO this simply yields the normal "spendable next block" behavior; for a positive `D` it yields `D` blocks *after* confirmation.

In other words: this is **exactly the BIP-68 relative-height evaluation** — a `D` -block relative height lock on an outpoint confirmed at height `c` is satisfied when the reference height (the height of the block *before* the spending block, i.e. `H-1`) satisfies `H-1 >= c + D`, i.e. `H >= c + D + 1`. VSD makes this **mandatory and value- and age-keyed** at the consensus layer rather than opt-in via the `nSequence` field. The `nSequence` field is **not** consulted for this check: even a sender who sets `nSequence = 0` on every input cannot bypass the delay on a large-value UTXO.

Base VSD (v1.0) as a special case. Setting `W(age) ≡ 1` for all ages, `F = 0`, and `D_cap = ∞` reduces `D(v, age)` to `Dv(v)` — the v1.0 value-only delay. The v1.0 specification is therefore a strict special case of v2.0; a deployment that adopts v1.0 semantics is a deployment of v2.0 with the age discount disabled.

Value-delay function `Dv(v)` (proposed starting values — tunable)

UTXO value	`Dv(v)` (blocks)	Approx. time
`< 1 BTC` (value < 100,000,000 sats)	0	none (no impact)

UTXO value	`Dv(v)` (blocks)	Approx. time
`1 – 10 BTC`	144	~1 day
`10 – 100 BTC`	4320	~30 days
`100 – 1000 BTC`	10080	~70 days (~2.3 months)
`>= 1000 BTC`	20160	~140 days (~4.7 months)

These are **draft values** for discussion. The exact thresholds and delays are consensus parameters (see "Open parameters") and should be conservative at activation: the goal is "large-only, moderate delay" at first, letting the economy self-adjust.

Age-discount factor `W(age)` (proposed starting values — tunable)

UTXO age (blocks)	`W(age)`	Approx. age in time
`< 144`	0.05	< 1 day (fresh)
`144 – 10,079`	0.15	1 – 7 days
`10,080 – 40,319`	0.35	7 – 28 days
`40,320 – 525,599`	0.65	28 days – 1 year
`>= 525,600`	1.00	>= 1 year (mature)

Notes: - `W` is monotonic non-decreasing: a *younger* UTXO never gets a *larger* discount. - The step boundaries are draft and tunable; the community may smooth them (e.g. a linear ramp) or adjust the number of tiers during review. - `W = 1.00` for mature UTXOs means the age discount is *zero* for the highest-risk category (dormant, long-idle UTXOs) — the full tier delay applies.

Composite delay `D(v, age)` — 2D reference matrix

The full delay matrix for the draft parameters (`T = 1 BTC` , `F = 144` , `D_cap = 20160`):

Value tier \ Age tier	Fresh <144 (W=0.05)	1d–7d (W=0.15)	7d–28d (W=0.35)	28d–1y (W=0.65)	≥1y (W=1.00)
1 – 10 BTC (Dv=144)	144 (1.0d) [1]	144 (1.0d) [1]	144 (1.0d) [1]	144 (1.0d) [1]	144 (1.0d) [1]
10 – 100 BTC (Dv=4320)	216 (1.5d)	648 (4.5d)	1512 (10.5d)	2808 (19.5d)	4320 (30d)
100 – 1000 BTC (Dv=10080)	504 (3.5d)	1512 (10.5d)	3528 (24.5d)	6552 (45.5d)	10080 (70d)
≥1000 BTC (Dv=20160)	1008 (7d)	3024 (21d)	7056 (49d)	13104 (91d)	20160 (140d)

[1] `F = 144` binds in this row: `Dv × W` ranges from 7 to 144, all ≤ 144, so the floor dominates and the delay is constant at 144 blocks (1 day) regardless of age. This is the intended behavior: *any* UTXO ≥ 1 BTC is delayed by at least 1 day.

Key comparisons (1000 BTC tier): - Base VSD (v1.0, no age): `D = 20160` blocks ≈ **140 days (~4.7 months)**. - VSD v2.0, fresh (age < 144): `D = 1008` blocks ≈ **7 days**. - VSD v2.0, mature (age ≥ 1y): `D = 20160` blocks ≈ **140 days** (unchanged

from v1.0). - **Speedup for active holders: ~20×** (20160 / 1008). - **Protection for dormant UTXOs: unchanged** (the full tier delay still applies).

Reference height: block vs. mempool

- **Block validation:** `H` is the height of the block being validated. The UTXO's age is `H − outpoint_height`. If any input fails the rule, the block is rejected.
- **Mempool:** `H` is the current chain tip height. A transaction whose any input is not yet "matured" (tip height < required) is **held in the mempool as non-mineable** — it is accepted for relay/storage but not eligible for inclusion until the tip reaches the required height. Because the UTXO's age *increases* as the tip advances, the mempool must **re-evaluate maturity periodically** (at least once per new block) for held transactions: as the tip advances, the age grows, `W(age)` may step up, and `D` may increase, so the `required` height may move forward. (In practice this is a non-issue: `D` is non-decreasing in age, so `required` is non-decreasing in time; a transaction that is not yet matured will only become *later* matured, never earlier. The mempool can cache the last computed `required` and re-check against the current tip.)

Unconfirmed previous outputs

If the referenced previous output is not yet confirmed (i.e. the transaction is a child of an in-mempool parent, as in CPFP), the input is **not** subject to the value-delay, because `outpoint_height` is undefined for an unconfirmed UTXO. The value-delay is a property of *confirmed* large UTXOs: once such a UTXO is confirmed, any later spend of it is delayed. A child of an unconfirmed large-value parent inherits the parent's eventual constraints transitively (the parent, when it is confirmed, will itself have been subject to the delay).

Validation pseudocode

```
function input_is_pending_vsd(input, utxo_set, ref_height):
    # input.prevout = (txid, vout)
    prev = utxo_set.lookup(input.prevout)
    if prev is None:
        return false            # not a confirmed UTXO -> not subject to VSD
    if prev.value < T:
        return false            # below threshold -> no delay
    age = ref_height - prev.outpoint_height
    D = min(D_CAP, max(F, round(Dv(prev.value) * W(age))))
    required = prev.outpoint_height + D +    # earliest mineable height
    return ref_height < required   # True => spend not yet allowed

# Block validation: for every tx in block, for every input:
#     if input_is_pending_vsd(input, utxo_set, block_height): reject block

# Mempool admission: a tx is eligible for mining only if, for all inputs:
#     not input_is_pending_vsd(input, utxo_set, tip_height)
# (re-evaluate at least once per new tip for held txs; D is non-decreasing
#  in age, so a held tx's required height never moves backward)
```

Interaction with existing mechanics

- **RBF (BIP-125):** Compatible. Competing replacements reference the same UTXO and share the same `required` height; you may re-sign and replace during the window, but no replacement confirms before `required`.
- **CPFP / child-pays-for-parent:** Compatible. A child can pay the fee of a pending large-value parent, but the pair still cannot confirm before the parent's `required` height.
- **Fee market:** Fee no longer guarantees *early* confirmation for large-value UTXOs — it only affects inclusion order after maturation. This is the intended property: price buys priority *within* the window, not the *start* of the window.
- **Floor, not ceiling:** VSD is the *minimum* delay enforced by consensus. Holders may voluntarily impose a **longer** delay or additional spending conditions on their own UTXOs via existing `OP_CSV` / `OP_CLTV` and Taproot scripts. VSD never restricts a holder from making their own coins *harder* to spend than the rule requires.

Sweep behavior and its honest limit

A legitimate holder who wants to move a large *dormant* UTXO (e.g. to consolidate or to pay a large invoice) must spend it; that spend incurs the full dormant-age delay (the maximum for the tier), and the resulting child UTXOs are *fresh* (age 0) with a much shorter delay. This is the **sweep** operation, and it has an honest, documented limit:

- **What sweep preserves:** the *total* time from the original UTXO's confirmation to the final spend of the sweep's children is at least the original UTXO's `required` height. The attacker (or any observer) cannot use a sweep to *shorten* the total time-to-spent for a given lineage of UTXOs.
- **What sweep does NOT preserve:** the *per-UTXO* protection for the *next generation*. A fresh 1000 BTC child produced by sweeping a dormant 1000 BTC UTXO has a delay of only ~7 days (vs. ~4.7 months for the parent). If the key is compromised *after* the sweep, the attacker faces only the fresh-child delay.
- **Why this is acceptable:** the sweep operation is performed by the *legitimate* holder (who controls the key), and the attacker's ability to exploit the shortened child delay is bounded by the fact that the holder has just demonstrated they are active and responsive (they just performed a sweep). The high-risk category (dormant, unresponsive holder) is not the one performing sweeps.
- **Follow-up (out of scope):** an "age inheritance" rule (a child UTXO inherits its parent's effective age) would close this gap; a concrete candidate for it — **Lineage-Age Inheritance (LAI)** — is specified, quantified, and stress-tested in §"Candidate Refinement (v2.1): Lineage-Age Inheritance (LAI)" below.

Grandfathering / activation behavior

The rule applies **only to UTXOs created at or after the activation block height**. Existing (pre-activation) UTXOs retain their current spend behavior for as long as they remain unspent.

Rationale: a soft fork must not retroactively change how the *existing* UTXO set is spent, or it would "freeze" legitimate cold holdings on a flag day with no warning — a far more disruptive behavior change. As the existing stock gets recycled (every confirmed spend re-creates its outputs post-activation), the protection **ratchets in gradually** over time.

Acknowledged gap: an *existing* large pre-activation UTXO can still be spent immediately by a key-holder. This is accepted as the standard soft-fork conservatism tradeoff; closing it fully would require a one-time consensus migration of the UTXO set, which this proposal deliberately avoids.

Deployment

Deployed as a **soft fork via BIP-9 version bits** (current version-bits implementation of BIP-9).

Version-bit assignment

The `version` field is a 32-bit integer; for version-bits signaling the base is `0x20000000` and signal bits occupy the low bits (bits 28–31 are reserved for the version number). On mainnet, **bit 0** has been used (BIP-68 CSV / BIP-141 SegWit) and **bit 1** has been used (BIP-341/342/343 Taproot). This deployment uses the **lowest unused bit — bit 2**:

bit	version value	prior mainnet use
0	`0x20000001`	BIP-68, BIP-141
1	`0x20000002`	BIP-341/342/343 (Taproot)
2	**`0x20000004`**	**this deployment (VSD)**
3	`0x20000008`	(free)

BIP-9 parameters (draft)

Parameter	Value	Note
deployment name	`vsd`	internal identifier
version bit	**2**	signal `0x20000004`
`nWindow`	**2016**	1 week of 10-min blocks (standard)
`nThreshold`	**1916**	95% of window (standard)
`nStart`	**TBD**	Unix epoch set only after signet + testnet4 validation completes
`nTimeout`	**TBD**	`nStart` + 31,557,600 s (1 Julian year) by default
`min_activation_height`	derived	per BIP-9: the first 2016-block boundary ≥ `nStart` at which the threshold can first be met

Concrete draft shape (epoch values are placeholders pending review + testnet completion; the *structure* is fixed): - `nStart` — a date ~12–18 months out, chosen so at least one full signal window elapses after it before the first eligible boundary. - `nTimeout` = `nStart` + 31,557,600 s. - If the 1916-threshold is met within a window whose start ≥ `nStart` and < `nTimeout`, the deployment activates at the **next 2016-block boundary**. If not met by `nTimeout`, it locks out (fail-safe, per BIP-9).

Consensus activation (this rule): enforced only for blocks at/after the BIP-9 activation block, and (per grandfathering) only for UTXOs created at/after that block. Non-upgraded nodes treat VSD-constrained large-value spends as valid; upgraded nodes treat early spends as invalid. The chain built by the upgraded majority remains valid to non-upgraded nodes, preserving the soft-fork property.

Policy/relay: the mempool "hold-not-mineable" behavior (accept + store, but not include in a block until matured) is a **policy** matter, not part of the activation consensus. Default policy: accept and hold; do not relay as mineable until matured. The periodic re-evaluation of held transactions (required because `D` depends on the tip height) is also a policy matter.

No block-size or data-format change. The rule consumes no extra bytes; it only references the existing `value`, `outpoint_height`, and the current chain tip (all already present in the UTXO set and block context).

Deployment path

1. **Signet / testnet4 first** — full validation on a developer signet (BIP-325-style) and testnet4, with mining, wallet (maturity-countdown UI), and exchange monitoring exercised against the §Test Vectors. No mainnet commitment yet.
2. **Reference implementation** — BIP-2-compliant patch to Bitcoin Core (consensus maturity check in block/tx validation + mempool hold-not-mineable policy with periodic re-evaluation) plus the §Test Vectors as unit/regression tests.
3. **Mainnet via BIP-9** — signal `0x20000004`, window 2016, threshold 1916, `nStart` / `nTimeout` as above. Grandfathering means only UTXOs created at/after the activation block are constrained, so activation is low-disruption.

Activation should follow the standard BIP-2 / BIP-127 process: draft → community review → Core implementation + test vectors → deployment BIP with final `nStart` / `nTimeout`.

Test Vectors

Test vectors below exercise the value-and-age delay rule (Specification §rule + § `Dv` + § `W`). All values use the **draft** threshold `T = 1 BTC = 100,000,000 sats`, the **draft** `Dv` table, the **draft** `W` table, `F = 144`, and `D_cap = 20160`. Heights are absolute chain heights. Per the rule, the earliest mineable height is `required = outpoint_height + D(v, age) + 1` (the `+1` encodes "cannot spend the confirmation block itself"). A spend is includable **at** `required` and any later height.

Value-delay table `Dv` (reference)

UTXO value (BTC)	sats	`Dv(v)` (blocks)
< 1	< 100,000,000	0
1 – < 10	100,000,000 – 999,999,999	144
10 – < 100	1,000,000,000 – 9,999,999,999	4320
100 – < 1000	10,000,000,000 – 99,999,999,999	10080
>= 1000	>= 100,000,000,000	20160

Age-discount table `W` (reference)

UTXO age (blocks)	`W(age)`
< 144	0.05
144 – 10,079	0.15
10,080 – 40,319	0.35
40,320 – 525,599	0.65
>= 525,600	1.00

Value-tier & threshold cases (age = 0, fresh)

#	UTXO value	outpoint height	age	`Dv`	`W`	`D`	`required` = outpoint+D+1	spend at (H)	includable?	boundary note
1	0.5 BTC (50,000,000)	800,000	0	0	0.05	0	800,001	800,001	✅ yes	**below threshold T** — only the standard +1 applies; spendable next block, no VSD delay
2	0.99999999 BTC (99,999,999)	800,000	0	0	0.05	0	800,001	800,001	✅ yes	**just under T** (1 sat below 1 BTC) — still no VSD delay; confirms boundary is exactly at 1 BTC
3	1 BTC (100,000,000)	800,000	0	144	0.05	144^2	800,145	800,001	❌ no	**exactly at T (lower bound)** — VSD engages; a spend at the normal next-block (800,001) is now rejected

#	UTXO value	outpoint height	age	`Dv`	`W`	`D`	`required` = outpoint+D+1	spend at (H)	includable?	boundary note
4	1 BTC (100,000,000)	800,000	0	144	0.05	144^2	800,145	800,144	❌ no	1 block **before** maturity — not mineable (mempool-held)
5	1 BTC (100,000,000)	800,000	0	144	0.05	144^2	800,145	800,145	✅ yes	**maturity boundary** — includable exactly at `required`
6	10 BTC (1,000,000,000)	800,000	0	4320	0.05	216	800,217	800,216	❌ no	**1→10 tier boundary (lower bound of 10-BTC tier)**, 1 block early
7	10 BTC (1,000,000,000)	800,000	0	4320	0.05	216	800,217	800,217	✅ yes	maturity boundary for 10 BTC tier (fresh)
8	9.99999999 BTC (999,999,999)	800,000	0	144	0.05	144^2	800,145	800,145	✅ yes	**just under 10-BTC tier** (1 sat below) — stays in 144-block tier (F binds), not 216
9	100 BTC (10,000,000,000)	800,000	0	10080	0.05	504	800,505	800,505	✅ yes	**10→100 tier boundary (lower bound of 100-BTC tier)**, at maturity (fresh)
10	1000 BTC (100,000,000,000)	800,000	0	20160	0.05	1008	801,009	801,009	✅ yes	**>=1000 top tier, fresh** — maturity boundary

#	UTXO value	outpoint height	age	Dv	W	D	required = outpoint+D+1	spend at (H)	includable?	boundary note
11	5000 BTC (500,000,000,000)	800,000	0	20160	0.05	1008	801,009	801,009	✅ yes	**top tier saturated, fresh** — delay equals the 1000-BTC fresh case; does not grow beyond D_cap

[2] In the `1 – 10 BTC` tier, `F = 144` binds: `Dv × W` ranges from 7 to 144, all ≤ 144, so `D = max(144, round(144 × W)) = 144` for all ages.

Age-tier cases (value = 1000 BTC, fresh outpoint)

These verify the `W(age)` step boundaries. All UTXOs are 1000 BTC (`Dv = 20160`), confirmed at height 800,000. `D_cap = 20160` , `F = 144` .

#	UTXO age (H – 800,000)	W	D = min(20160, max(144, round(20160×W)))	required	spend at (H)	includable?	boundary note
A1	72 (< 144)	0.05	1008	801,009	801,008	❌ no	**fresh tier**, 1 block before maturity
A2	72 (< 144)	0.05	1008	801,009	801,009	✅ yes	**fresh tier**, maturity boundary
A3	144 (= 144, 1d–7d lower bound)	0.15	3024	803,025	803,024	❌ no	**fresh→1d tier boundary (lower bound of 1d–7d)**: at age 144, W jumps 0.05→0.15; 1 block early
A4	144 (= 144)	0.15	3024	803,025	803,025	✅ yes	1d–7d tier, maturity boundary
A5	10,079 (1d–7d upper bound)	0.15	3024	803,025	803,025	✅ yes	1d–7d tier, at upper bound
A6	10,080 (= 10,080, 7d–28d lower bound)	0.35	7056	807,057	807,056	❌ no	**1d→7d tier boundary (lower bound of 7d–28d)**: at age 10,080, W jumps 0.15→0.35; 1 block early
A7	10,080 (= 10,080)	0.35	7056	807,057	807,057	✅ yes	7d–28d tier, maturity boundary
A8	40,319 (7d–28d upper bound)	0.35	7056	807,057	807,057	✅ yes	7d–28d tier, at upper bound

#	UTXO age (H − 800,000)	W	D = min(20160, max(144, round(20160×W)))	required	spend at (H)	includable?	boundary note
A9	40,320 (= 40,320, 28d–1y lower bound)	0.65	13104	813,105	813,104	❌ no	**7d→28d tier boundary (lower bound of 28d–1y)**: at age 40,320, W jumps 0.35→0.65; 1 block early
A10	40,320 (= 40,320)	0.65	13104	813,105	813,105	✅ yes	28d–1y tier, maturity boundary
A11	525,599 (28d–1y upper bound)	0.65	13104	813,105	813,105	✅ yes	28d–1y tier, at upper bound
A12	525,600 (= 525,600, ≥1y lower bound)	1.00	20160	820,161	820,160	❌ no	**28d→≥1y tier boundary (lower bound of mature)**: at age 525,600, W jumps 0.65→1.00; 1 block early
A13	525,600 (= 525,600)	1.00	20160	820,161	820,161	✅ yes	**mature tier**, maturity boundary — full tier delay
A14	1,000,000 (>> 1y)	1.00	20160	820,161	820,161	✅ yes	**mature tier saturated** — delay equals the 525,600 case; does not grow beyond D_cap

Floor-dominance case (1 – 10 BTC, all ages)

These verify that `F = 144` binds in the `1 – 10 BTC` tier for *all* ages (the `Dv × W` product never exceeds 144 in this tier).

#	UTXO value	outpoint height	age	W	Dv×W	D = max(144, round(Dv×W))	required	spend at (H)	includable?	note
F1	5 BTC (500,000,000)	800,000	72	0.05	7	144	800,145	800,144	❌ no	F binds (7 < 144); 1 block early
F2	5 BTC (500,000,000)	800,000	72	0.05	7	144	800,145	800,145	✅ yes	F binds; maturity boundary
F3	5 BTC (500,000,000)	800,000	500,000	0.65	94	144	800,145	800,145	✅ yes	F binds (94 < 144); same D as fresh

#	UTXO value	outpoint height	age	`W`	`Dv×W`	`D` = max(144, round(Dv×W))	`required`	spend at (H)	includable?	note
F4	5 BTC (500,000,000)	800,000	1,000,000	1.00	144	144	800,145	800,145	✅ yes	F and product both = 144; maturity boundary

Grandfathering cases (activation at height `A`)

These verify the rule applies **only** to UTXOs created at/after the activation block height `A`. Assume `A = 900,000`.

#	UTXO created at height	UTXO value	spend H	age	subject to VSD?	includable?	note
G1	850,000 (< A)	100 BTC	850,001	1	❌ no	✅ yes	**pre-activation UTXO** — exempt from VSD; spendable at the normal next block despite value
G2	900,000 (= A)	100 BTC	900,001	1	✅ yes	❌ no	**created exactly at activation** — subject to VSD (creation-height bound is inclusive); fresh age → D=504, required=900,505; a normal next-block spend is rejected
G3	900,001 (> A)	100 BTC	910,082	10,081	✅ yes	✅ yes	**created just after activation**; the delay is keyed on the UTXO's own creation height (900,001) and age (10,081 → W=0.35 → D=3528, required=903,529); at H=910,082 the UTXO is matured. (The spend at 903,528 would be ❌ not yet includable.)

Unconfirmed-previous-output case

#	prev out	spend H	includable?	note
U1	unconfirmed 50 BTC (in mempool)	tip 100,000	✅ yes	unconfirmed prev → not subject to VSD; child can be included (CPFP)
U2	unconfirmed 50 BTC parent; parent later confirms at 100,050	child re-spent at 100,051	governed by parent's VSD	the *confirmed* 50 BTC UTXO, when spent, is subject to Dv(50 BTC)=4320 (10–100 tier) with an age-dependent W; larger confirmed values follow the 2D table

Sweep cases (honest limit, informational)

These document the sweep behavior (§Specification, "Sweep behavior"). They are **informational** (they verify the honest limit, not a consensus rule violation).

#	parent UTXO	parent age at sweep	parent D	child UTXO	child age at re-spend	child D	note
S1	1000 BTC, confirmed at 500,000	400,000 (>> 1y)	20160	1000 BTC (sweep child), confirmed at 900,000	72 (fresh)	1008	**sweep shortens per-UTXO protection for the next generation**: the parent's 4.7-month delay becomes the child's 7-day delay. The *total* time from the parent's confirmation (500,000) to the child's spend (≥ 901,009) is still ≥ the parent's required height (520,161).
S2	100 BTC, confirmed at 800,000	100,000 (>> 1y)	10080	100 BTC (sweep child), confirmed at 900,000	72 (fresh)	504	Same pattern at the 100–1000 tier: 70-day parent delay → 3.5-day child delay.

Expected wallet/UI behavior (non-consensus, informational)

- A wallet sending from a fresh (age < 144) 1000 BTC UTXO confirmed at height 800,000 should display a **maturity countdown**: "spendable in 1,009 blocks (~7 days); target height 801,009" (801,009 = 800,000 + 1008 + 1).
- A wallet sending from a dormant (age ≥ 1y) 1000 BTC UTXO should display: "spendable in 20,161 blocks (~4.7 months); target height 820,161" (820,161 = 800,000 + 20160 + 1).
- A 0.5 BTC UTXO should show **no countdown** (below threshold).
- A 5 BTC UTXO (1–10 tier) should show a **constant 1-day countdown** regardless of age (F binds).
- The countdown must be driven by the **UTXO's own outpoint height, value, and current age**, independent of `nSequence`. For held (not-yet-matured) transactions, the wallet should update the countdown as the tip advances (age grows, `D` may increase).

Invariants the vectors must catch

1. **Threshold is value-keyed**: cases 1/2 (below T) vs 3 (at T) confirm the boundary sits at exactly 1 BTC — 1 sat below is only the standard next-block rule; 1 sat at/above triggers the VSD delay.
2. **Value-tier boundaries are lower-inclusive**: cases 3 (1 BTC), 6 (10 BTC), 9 (100 BTC), 10 (1000 BTC) confirm each value tier's delay applies from its lower value bound; case 8 (9.99999999 BTC) confirms the value just *under* a tier keeps the lower delay.
3. **Age-tier boundaries are lower-inclusive**: cases A3 (age 144), A6 (age 10,080), A9 (age 40,320), A12 (age 525,600) confirm each age tier's `W` applies from its lower age bound; the case just *below* each boundary keeps the lower `W`.
4. **Monotonic in age (discount)**: for a fixed value, `D(v, age)` is non-decreasing in age — a *younger* UTXO never gets a *longer* delay. Cases A2 (age 72, D=1008) → A4 (age 144, D=3024) → A7 (age 10,080, D=7056) → A10 (age 40,320, D=13104) → A13 (age 525,600, D=20160) confirm the monotonic steps.
5. **Monotonic in value (delay)**: for a fixed age, `D(v, age)` is non-decreasing in value — a *larger* UTXO never gets a *shorter* delay. Cases 3 (1 BTC, D=144) → 7 (10 BTC, D=216) → 9 (100 BTC, D=504) → 10 (1000 BTC, D=1008), all at age 0, confirm the monotonic steps.
6. **Maturity is inclusive**: cases 4/5 (1 block early → rejected; exactly at `required` → accepted) and A1/A2 confirm the boundary is `H >= required`, not `H > required`.

7. **Floor dominance in the 1–10 tier**: cases F1–F4 confirm that `D = 144` for *all* ages in the 1–10 BTC tier (the floor binds because `Dv × W ≤ 144` throughout this tier).
8. **Cap saturation**: cases 11 (5000 BTC, fresh) and A14 (1000 BTC, age >> 1y) confirm `D` does not grow beyond `D_cap = 20160`.
9. **Grandfathering is height-keyed**: G1 vs G2 confirm pre-activation UTXOs are exempt, activation-height UTXOs are not.
10. **Unconfirmed prevs are exempt**: U1 confirms an in-mempool large-value UTXO is not VSD-gated until confirmed.
11. **Sweep honest limit (informational)**: S1/S2 document that a sweep shortens the *next-generation* per-UTXO delay but does not shorten the *total* time-from-original-confirmation-to-final-spend.

These vectors are intended as the Core `functional` / `unit` test suite for the consensus check in §Validation pseudocode.

Backward compatibility

- **Soft fork (backward-compatible to old nodes).** The new validity set ⊆ old validity set. Old nodes accept the chain; they simply do not relay/mine certain early large-value spends. Standard BIP9 soft-fork caveats apply (old nodes cannot distinguish "invalid under new rules" from "not included"; the upgraded majority governs consensus).
- **No data migration.** Existing UTXOs, addresses, and wallet files are untouched (grandfathering).
- **No client-format change.** `value`, outpoint metadata, and the current chain tip already exist in the UTXO set and block context.
- **Held transactions:** wallets must handle "large-value tx accepted but pending maturation" gracefully (show a maturity countdown that *updates as the tip advances*), analogous to existing CSV-maturity display but with an age-dependent target height.

Known limitations and non-goals

This proposal is deliberately scoped to *partially* improve security *without* disruptive changes. It **does not** and is **not intended to**:

1. **Prevent theft from a compromised key.** It lengthens the window and enables the holder to race; it does not make the theft impossible. An offline or unaware holder can still lose funds after the delay.
2. **Protect addresses with no living holder** (lost keys, Satoshi's P2PK). For these, VSD only buys *observation* time for the community. Genuine protection there would require a separate community-emergency mechanism (out of scope; that is a governance/centralization-sensitive change this proposal avoids).
3. **Replace root-cause fixes.** Key-hiding outputs (P2WPKH/P2TR/P2MR) and post-quantum signatures remain the primary defenses. VSD is an additive behavior-layer time window layered beneath them.
4. **Handle HD-wallet key chains.** Racing one UTXO does not secure the rest of a key; key rotation is still required.
5. **Be tamper-proof against an attacker who simply waits.** If the attacker waits out the delay and the holder never responds, the theft completes. The delay converts "instant" into "bounded-time", not into "impossible".
6. **Be tamper-proof against a same-key attacker who *ages* a UTXO.** A same-key attacker cannot make a UTXO *younger* (the age is derived from the immutable confirmation height), but they *can* perform a sweep: spend the

dormant UTXO (incurring the full dormant-age delay) and re-create fresh children with a shorter delay. If the key is compromised *after* the sweep, the attacker faces only the fresh-child delay. This is the **sweep limit** (§Specification, "Sweep behavior"); it is accepted because the sweep is performed by the legitimate holder and the shortened child delay is a bounded concession for the next generation.

7. **Distinguish an active holder's fresh UTXO from an attacker's fresh large UTXO.** A fresh 1000 BTC UTXO that the *legitimate* holder just re-spent is indistinguishable, at the consensus layer, from a fresh 1000 BTC UTXO that an *attacker* just created (e.g. by sweeping a stolen dormant UTXO). Both get the fresh-age delay (~7 days). The age discount therefore *cannot* provide full protection against an attacker who performs a sweep immediately after a key compromise; it provides protection against an attacker who *waits* for a UTXO to mature before spending it. This is an inherent limitation of using UTXO age as the discount signal.

Non-goals: no new address type, no new output format, no new signature scheme, no new opcode, no on-chain governance, no change to the data model, no change to block size.

Candidate Refinement (v2.1): Lineage-Age Inheritance (LAI)

> **STATUS: CANDIDATE — not yet normative.** This section proposes, quantifies, and stress-tests a refinement to the v2.0 age rule. It is **not** part of the normative specification: the v2.0 rule (§Specification, "The rule") remains the baseline. LAI is presented as a *candidate* with its exact mechanism, its measured costs, and the open questions it leaves for community decision, so reviewers can evaluate it *alongside* v2.0 rather than in isolation. If adopted in a later version, it would replace the `age(0, H)` input to `D` with a lineage-inherited `eff_age`.

Why v2.0 leaves a hole: the sweep attack

[EN] v2.0 keys the discount on a UTXO's *own* confirmation age. That is tamper-resistant against a *single* spend (an attacker cannot make a UTXO younger), but it is **not** tamper-resistant against *repeated* spends. An attacker who has stolen a *dormant* large UTXO can **re-sweep** it: spend the parent (paying the parent's full dormant-age delay) and create a *fresh* child (own-age 0). The child now sits in the **fresh tier** (`W = 0.05` , `D ≈ 1008` blocks ≈ 7 days for the ≥1000 BTC tier). By sweeping every few blocks, the attacker can keep the *active* UTXO perpetually fresh, so the **140-day dormant protection is collapsed to a 7-day protection by a single sweep**.

v2.0's sweep invariant ("total time from the original confirmation to the final spend ≥ the original `required` height") still holds — **but only for the lineage as a whole**. The per-UTXO protection of the *next generation* is exactly what a sweep resets. This is the hole LAI closes.

The core idea: make age lineage-inherited and monotone

[EN] LAI redefines the age that `D` consumes. Instead of a UTXO's own confirmation age, it uses an **effective age** `eff_age` that is *monotone non-decreasing* down the spend lineage and *saturates* at a fixed point:

```
AGE_MATURE = ,  ( year; the point where W = ., i.e. the tier delay has saturated)

eff_age(C) =
    min(AGE_MATURE, own_age(C))                       if C has no in-scope parent (root / pre-activation)
    min(AGE_MATURE, max(own_age(C), eff_age(parent)))  otherwise

D(C) = D(v, eff_age(C))      (the v2. delay function, fed eff_age instead of own age)
```

Because `eff_age(C) >= eff_age(parent)` whenever a parent exists, a child can **never be younger (for delay purposes) than its parent**. An attacker cannot reset a mature lineage to "fresh" by sweeping — every child inherits the parent's mature effective age.

Why saturate at `AGE_MATURE` (load-bearing): without saturation a long-lived lineage's effective age would grow without bound and the `D_cap` would simply bind; the rule would still be well-defined but the "age" value would be unbounded and harder to reason about. Saturating at the `W = 1.0` point (1 year) keeps the mature tier finite, makes `eff_age` bounded, and guarantees the delay is exactly the tier cap once a lineage has been dormant a year or more — which is the intended semantics ("dormant ≥ 1 year pays the full delay").

How LAI was arrived at (design evolution)

[EN] - **Candidate A — "cap the inherited age at `D_cap`."** Rejected. Capping the *age* at 20,160 (a *delay* number) is a category error: `W(20,160) = 0.35`, so the delay would be `round(20160 × 0.35) = 7,056` blocks — **not** the full tier delay. The cap must be an *age* at which `W = 1.0`, i.e. `AGE_MATURE = 525,600`, not a delay. (Verified: capping age at `D_cap` gives 7,056; capping at 525,600 gives 20,160.) - **Candidate B — "fractional inheritance" (child inherits a fraction of the parent's age).** Rejected. An arbitrary fraction is not principled, and — more importantly — it still leaves a *partial* reset path (a child inherits only 50% of the parent's age, so enough re-sweeps drive the inherited age down). The monotone `max` with saturation has no such decay: it is a floor, not a discount. - **Candidate C — "lineage min-height" (child's age measured from the lineage's earliest outpoint).** This is equivalent to monotone-`max` for single-parent chains and is the intuitive formulation; it is what the `root_height` recursion below computes. Adopted, with saturation.

A concrete threat, walked through (the high-frequency re-sweep)

[EN] 1. An attacker steals a **dormant 1,000 BTC UTXO** whose root was confirmed 1 year ago (height `H0`; `own_age` at theft ≈ 525,600). 2. **Under v2.0 (no LAI):** the attacker sweeps → a fresh child (own-age 0, `W = 0.05`, `D = 1,008` ≈ 7 days). Re-sweeping every few blocks keeps the active UTXO perpetually fresh. **The 140-day dormant protection collapses to 7 days after one sweep.** 3. **Under LAI:** the child's `eff_age = min(525,600, max(1, 525,600)) = 525,600` → `D = 20,160` (140 days). **The sweep does not reset the age.** A multi-generation chain (`S0 → S1 → S2 → …`, each own-age ≈ 1) inherits the mature effective age at every hop; there is no reset. (Verified, property P4 below.)

What LAI costs: the honest tradeoff

[EN] LAI is not free. Its marginal cost, measured against v2.0, depends on the legit holder's pattern:

Legit-holder pattern	v2.0 cost	LAI cost	Marginal (LAI − v2.0)
A. sweep a dormant 1-year 1,000 BTC UTXO → move to a cold wallet → **sit** (do not spend the child)	parent pays 20,160 (≈140d)	parent pays 20,160 (≈140d)	**0** — inheritance only bites on the *child-spend* step
B. sweep a dormant 1-year UTXO → **immediately spend the child** (consolidate & pay)	20,160 + 1,008 = 21,168 (≈147d)	20,160 + 20,160 = 40,320 (≈280d)	**+19,152 blk (≈133d)**
C. active holder, fresh parent → fresh child	1,008 (≈7d)	1,008 (≈7d)	**0** — fresh tier preserved

The read: LAI is **free for the two most common legit operations** (active holders, and "sweep to cold and park"), and its **only cost is Pattern B** — a holder who sweeps a *dormant* large UTXO and *immediately* spends the resulting child pays an extra ~4.4 months. That is the price of closing the sweep hole, and it is precisely the "behavior-layer, partial, non-disruptive" tradeoff this proposal is built on: LAI *improves* the highest-risk case (dormant stolen UTXO) at a *bounded, well-characterized* cost to one specific legit operation. The community must decide whether that price is acceptable; that decision is the point of presenting it as a **candidate**.

Invariants LAI must preserve (and the vectors that check them)

[EN] 1. **P1 — Fresh tier preserved:** fresh parent ⇒ fresh child (`D = 1,008`). *Legit active holders are unaffected.* 2. **P2 — Full delay preserved:** mature parent (≥ 1 year) ⇒ mature child (`D = 20,160`). 3. **P3 — Proportional:** a 6-month parent ⇒ a 6-month-tier child (`D = 13,104`). 4. **P4 — No reset:** a multi-generation re-sweep of a mature lineage cannot reduce the child's `eff_age` below the root's (verified: `S0→S1→S2` , each own-age ≈ 1, all inherit `eff_age = 525,600` , `D = 20,160`). *This is the property that closes the v2.0 hole.* 5. **P5 — Monotone:** for any child with an in-scope parent, `eff_age(child) >= eff_age(parent)` . 6. **P6 — Park is free:** sweeping a dormant UTXO and *not* spending the child incurs **no** marginal delay under LAI (the inheritance is only evaluated when the child is itself spent).

Test vectors (LAI candidate)

[EN] Values use the draft `Dv` , `W` , `F = 144` , `D_cap = 20,160` , `AGE_MATURE = 525,600` , and a 1,000 BTC UTXO unless noted. `own_age` is the UTXO's own confirmation age; `eff_age` is the LAI lineage-inherited age; `D` is the resulting delay.

#	Pattern	root own-age	child own-age	v2.0 D (own-age)	LAI eff_age	LAI D	Invariant
L1	fresh parent → fresh child	72	1	1,008	72	1,008	P1
L2	mature parent → fresh child	525,600	1	1,008	525,600	20,160	P2
L3	6-month parent → fresh child	262,080	1	1,008	262,080	13,104	P3
L4	re-sweep chain (S0→S1→S2), each own-age ≈ 1, S0 mature	525,600	1 (×3)	1,008	525,600	20,160	P4
L5	monotone: child own-age < parent eff_age	131,040	10	1,008	131,040	13,104	P5
L6	sweep dormant, **do not spend child**	525,600	— (unsent)	parent 20,160	n/a	parent 20,160	P6

#	Pattern	root own-age	child own-age	v2.0 D (own-age)	LAI eff_age	LAI D	Invariant
L7	pre-activation root → post-activation child (grandfathering limit)	pre-A	1	1,008	1	1,008	limit

What LAI does NOT fix (honest limit)

[EN] - **Pre-activation roots are not protected by LAI.** If a lineage's root is a *pre-activation* (grandfathered) UTXO and the attacker sweeps it, the child's parent is out-of-scope (pre-activation), so `eff_age` falls back to the child's **own** age → fresh. LAI closes the hole only for lineages whose *root is post-activation*. This is the v2.0 grandfathering limit, now applied one generation deeper. (Vector L7.) - It does **not** fix the "attacker waits out the delay" case — if the holder never responds, the theft completes after the (now lineage-inherited) delay. - It **raises the mempool implementation cost**: `eff_age` depends on the *lineage*, not a single outpoint, so the mempool must track parent `eff_age` values for held transactions (a real but bounded cost). This is the main reason LAI is a **candidate** rather than a folded-into-v2.0 rule.

Open parameters (LAI)

[EN] - **Whether to adopt LAI at all.** It is a candidate; the v2.0 normative baseline stands on its own. - `AGE_MATURE` — the saturation point (draft: 525,600 / 1 year). - **Root definition in multi-parent spends.** The recursive `eff_age(C) = min(AGE_MATURE, max(own_age(C), eff_age(parent)))` assumes a single in-scope parent. In a CPFP or a multi-input sweep, a UTXO can have several parent outpoints; the precise rule is `eff_age(C) = min(AGE_MATURE, max(own_age(C), max over in-scope parents of eff_age(parent)))`. This UTXO-graph definition needs to be pinned down before LAI could be normative. - **Interaction with mempolicy periodic re-evaluation:** `eff_age` is a function of the lineage and the tip; the periodic re-evaluation the v2.0 mempool already performs must be extended to recompute `eff_age` for held (not-yet-mature) transactions.

Open parameters

To be finalized during review (each is a tunable consensus/policy parameter):

- `T` — activation threshold (draft: 1 BTC).
- The value-delay table `Dv(v)` — tier boundaries and per-tier block counts.
- The age-discount table `W(age)` — tier boundaries, per-tier discount factors, and the number of tiers (or a continuous alternative such as a linear ramp).
- `F` — the floor (draft: 144 blocks). Whether it should be a single scalar (current) or a per-value-tier vector.
- `D_cap` — the cap (draft: 20160 blocks). Whether a *higher* cap is needed for extreme values, or whether the cap should be *lowered* to reduce the dormant-UTXO delay.
- Whether the delay should be **linear in log(value)** rather than a step function (a continuous alternative for the value axis, independent of the age axis).
- Grandfathering: strictly "created at/after activation" vs. a bounded transition window for pre-activation UTXOs.
- **Age inheritance (LAI)**: now specified as a *candidate* in §"Candidate Refinement (v2.1)"; not yet normative. Open questions: whether to adopt it, `AGE_MATURE`, and the multi-parent root definition.

- Mempool policy defaults (hold vs. reject), the periodic re-evaluation frequency for held transactions, and whether to surface a standard "maturity height" field to relays.

References

- BIP-68: Relative lock-time (OP_CSV) — the primitive this builds on.
- BIP-65: CheckLockTimeVerify (OP_CLTV) — absolute lock-time, related semantics.
- BIP-141/143: SegWit — UTXO/witness set and outpoint metadata.
- BIP-341/342/343: Taproot — existing key-hiding and script-path outputs.
- BIP-9: Version bits-based consensus deployment.
- BIP-125: BIP125-replaceability signaling (RBF).
- BIP-360 (Pay-to-Merkle-Root) / BIP-361 (Post Quantum Migration and Legacy Signature Sunset) — the root-cause quantum-defense layers this proposal complements.
- BIP-2: BIP process and authorship guidelines.

Copyright

This BIP is licensed under the Creative Commons CC0 1.0 Universal Public Domain Dedication.